INF264 Project 1

1. Introduction

This is a project dedicated to creating a decision tree model used to predict whether or not a customer will “Churn” (Leave). It consists of 4 files. A readme document to explain how to run the project, the data in the churn-data.csv file, the decision_tree.py file to build the model itself and the run_experiments file to train the model and analyse the results.

2. Implementation

2.1 Structure of the code

The code for this project is structured to make it as interpretable and readable as possible. In order to do this, I have tried to take advantage of helper-methods in as many ways as possible. Examples if this is the fact that the build_tree method is separate from the rest. This makes it easier to tell what is being done at each point of the program. The same idea is being used when making the node its own class.

All the helpers that I have used are:

- **_impurity_from_counts:** Computes the impurities from the class count
- **Impurity**: calculates the impurity
- **_Information_gain:** uses the impurities to calculate the information gain at a certain step
- **_best_split**: finds the best pair of feature and threshold to maximize the information gain
- **_most_common_label:** Finds the most common label (self explanitory)
- **_build_tree**: combines all this to build the actual decision tree.
- **_predict_one:** gives the output for the model of a single x
- **Permutation_importance:** returns the importance of each feature.

2.2 The ID3 algorithm

ID3 (Iterative Dichotomiser 3) builds a decision tree by repeatedly asking: *"which single feature, if I split on it right now, does the most to separate the classes?», and* then just doing that, node by node, until it runs out of reasons to keep splitting. This is a greedy algorithm, meaning that it only looks one step ahead to see what its best choice is. This is a computationally cheap way of doing things, but it does not guarantee the smallest or most accurate tree.

The model stops on 4 different occasions:

- **When all the labels are identical:** The node is pure. Impurity is already 0, so no split could improve anything. The leaf stores that label directly rather than going through _most_common_label, since there's only one.

- **All feature vectors identical:** Every remaining sample has the same values across every feature, but the labels disagree. No test on any feature can separate them, so the best you can do is predict the majority. Without this check, _best_split would find no valid threshold and you'd have nothing to recurse on.
  The expression reads inside-out: X == X[0] compares every row against the first, .all(axis=0) collapses down each column to "is this column constant?", and the outer .all() asks whether that holds for all columns.
- **When max depth is reached:** This is when the models hyperparameter, made to stop the model from overfitting kicks in. The model has exceeded the specified max depth and will then stop.
- **No useful split available:** Two distinct situations collapsed into one branch:
  - feature_index is None,_best_split found no valid threshold on any feature. Every feature is constant at this node even though the rows aren't fully identical (possible when some column is constant here but the node was reached without triggering condition 2. In practice condition 2 catches most of these, so this is belt-and-braces).
  - gain <= 0 — a split exists but doesn't reduce impurity.

The **recursion** of the model works in two ways. It either hits one of the base cases mentioned above and creates a leaf, or it creates two new children, which are themselves built by _build_tree. This leads to the data in each node shrinking and the depth of the model increasing. The predict method uses **recursion** as well. It uses the same patter, but for a different purpose. It moves down the tree to predict instead of building.

2.3 Impurity measures

We give two options to use two different impurity measures.

Entropy:

$$H(D) = -\sum_{k} p_k \log_2 p_k$$

GINI:

$$G(D) = 1 - \sum_{k} p_k^2$$

The recursion never computes entropy directly, so swapping criteria touches one function. This is how we made it possible for out model to choose between the two options. You simply select either entropy or GINI as a hyperparameter and train the model using that.

2.4 Choosing split thresholds

For this project I chose to evaluate every midpoint between consecutive distinct values of every feature, which is the most thorough option. The reason we did this was to account for the different distributions of the variables. Some were unproblematic like the 0/1 columns but others like the continuous ones were bound to give some issues with the standard approach. This is because there could be many possible splits, and the mean/median-only version tested exactly one of them. The mean is an arbitrary point with no reason to be near the optimum. Our solution fixed this problem for all the variables, at the cost of extra computing.

I had a few different options on how to do this. The obvious way to do this was to loop over each candidate threshold, build a boolean mask, compute impurity is O(n) per candidate. which would be unusably slow.

Instead, I use a **cumulative-sum sweep**. For each feature:

1. Sort the samples by that feature once (argsort).
2. One-hot encode the labels and take a cumulative sum down the sorted order. Row *i* of that cumsum is exactly the class counts to the left of a cut after position *i*, so one cumsum gives you the left-side counts for *all* candidate splits simultaneously.
3. Right-side counts are just total, left, by subtraction.
4. Compute impurity for every candidate at once, vectorized over the whole stack.

Cost per feature drops to O(n log n), dominated by the sort, regardless of how many thresholds there are.

2.5 Verifying correctness

To verify correctness, I used a couple of different techniques. One of the techniques that I used was to create a dataset where I could know what the model should weigh and not.

When I used these settings for the make_classification method:

```
X, y = make_classification(
    n_samples=3000, n_features=5,n_redundant=0, n_informative=3,
    n_classes=2, random_state=42, shuffle=False
),
```

I knew that it should weigh the 3 first variables heavy and not look at the two last variables. The output from the permutation_importance method was: [0.32316667 0.05633333 0.1245 0. 0.], which matched my prediction.

I also looked at how my model compared to the sklearn version. Our model behaves in exactly the same way as the sklearn version. The results were the same up to four decimals. This is a good sign for my model being implemented correctly.

---

3. Model selection

3.1 Which hyperparameters

For the sake of this assignment, I only had two hyperparameters for the sake of simplicity, max depth and criterion. This made the computations needed for running the programs easier. If I had more time and resources, I could have looked at more hyperparameters like min_samples_leaf, min_samples_split, min_impurity_decrease or class weighting.

3.2 Which values

When I chose which values to train on the criterion was easy: GINI and entropy, but when it came to the max depth values, it got more complex. I chose the values [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 20, None]. The values get more sparse the higher they get. This is because of the diminishing returns that you get with a higher max depth, and given our limited sample size, i did not want to go beyond 20. I did test the model out with no limit as well, even though chances were high for overfitting.

3.3 Which method

The method for validating the models I used was a 5-fold stratified cross-validation. We did this to decrease the chance of a model getting lucky with a good validation set, since the max over N noisy estimates is biased upward. The problem does not disappear with CV, but it is an improvement. CV cuts the noise by $1/\sqrt{k}$, which in this case Is $1/\sqrt{5}$. I used stratified data to avoid the churn rate from being too varied per fold and create a noisier F1 score.

3.4 Which performance measure

Seeing that the churn rate is about 26%, the model could learn to predict everything as “no churn” and get an accuracy of 73%. This is not what we want from our model. Because of this we used F1 as our performance measure instead. The F1 harmonic mean is dragged down by whichever of precision and recall is worst, so it can’t be gamed in the same way that accuracy can. This is also why our depth-1 tree scored exactly 0.000 while accuracy was 0.73. We logged accuracy and balanced accuracy as well as the F1 score for deeper insights.

3.5 Fairness / unbiasedness

To ensure fairness and unbiasedness I made sure that any data used to make a decision no longer could serve as an estimate of generalization. This covers every part of the project. I made sure that the test set was separate from the training data before I started anything else, to avoid data leakage. It is also important to mention that even with the choice to use CV instead of holding of data, the scores are still biased upward, so we would expect the real scores to be lower, which we will see when the model is evaluated later. It is also worth noting that all the data was already numeric, so there was no need for imputation, scaling or similar, so nothing is learned from the test set that can flow backward into training. The permutation importance part

of the project was done after the test score is reported and feeds back into nothing. I never dropped any features based on these results to retune. My model is in no way 100% unbiased, but I did what I could to get an as fair and unbiased model with the time and resources available.

3.6 Reproducibility

To ensure reproducibility, I used a random state throughout the code wherever possible, so that it would give the same result every time. I also printed out my library versions which are bellow. There is no randomness that should make our program give any different results no matter how many times you were to run it, as long as the library versions are the same.

Versions and seed	***for reproducibility***
random seed	42
python	3.12.12
numpy	2.3.1
pandas	2.3.3
scikit-learn	1.8.0

3.7 Results

Criterion	***Max depth***	***CV accuracy***	***CV F1***	***CV F1 std***	***CV balanced acc***
gini	2	0.7454	0.6016	0.0121	0.7383
entropy	2	0.7454	0.6016	0.0121	0.7383
entropy	7	0.7936	0.5843	0.0128	0.7148
gini	7	0.7904	0.5827	0.0128	0.7141
entropy	6	0.7932	0.5786	0.0341	0.7118
gini	6	0.7906	0.5752	0.0320	0.7099
entropy	5	0.7911	0.5623	0.0208	0.6996
gini	5	0.7892	0.5579	0.0204	0.6970
gini	3	0.7787	0.5561	0.0152	0.6969
entropy	3	0.7787	0.5561	0.0152	0.6969

From our accuracy and F1 vs depth curves, we can see that the F1 score peaks at 2, while accuracy peaks at around 5-7, our plotted metrics disagree. This is because shallow trees often

predict churn liberally with high recall but lower precision. deeper trees have smaller purer leaves that flip to "no churn" leading to higher precision up but lower recall.

The difference between the CV F1 scores on max depth 2 and 7 are 0.018876. This is above the standard deviation we got for F1 of 0.0121. Because of this I would say that the depth 2 model is a clear winner given its better F1 and simplicity.

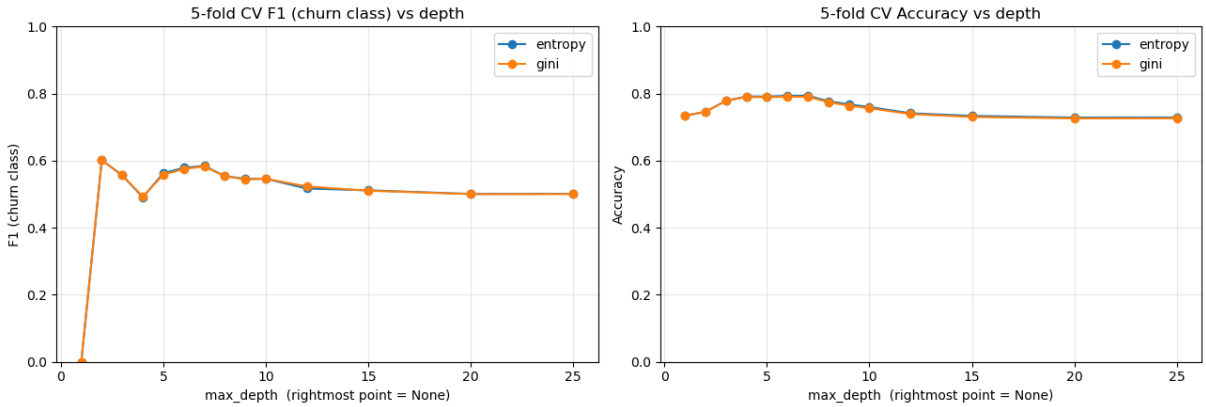


Figure 1: Accuracy and F1 scores against depth

Since both values for criterion are so similar (GINI being a first -order Taylor approximation of entropy), this hyperparameter likely does not matter as much as max depth. This is confirmed by the fact that they are basically identical on the plot above, the difference barely being visible at a few points.

CV results	
Criterion	Entropy
Max depth	2
CV Accuracy	0.7454
CV F1	0.6016
CV F1 std	0.0121

4. Evaluation and comparison to sklearn

Final model:

Metric	***Score***
Accuracy	0.7285
Balanced accuracy	0.7196
F1	0.5784

We got an accuracy of 0.7285, this is worse than a model always predicting “no-churn” with 0.735, but this is not necessarily a bad thing. Our balanced accuracy is 0.7196, which is way higher than the “no-churn” model would get with 0.5. The F1 score (the most important for my implementation) was 0.5784. It is a respectable score, especially given the simplicity of the model. It was also as predicted lower than our CV F1 score at 0.6016 with a drop of 0.023.

We also trained another model for comparison with sklearns build in decision tree. It was trained in the same way, with the same grid, the same folds and the same options for metrics. As you can see from our results it gave exactly the same scores. This is exactly what you should expect given the similarities in the way they were trained and the in algorithms themselves.

Sklearn model:

Metric	*Score*
Accuracy	0.7285
Balanced accuracy	0.7196
F1	0.5784

---

5. Feature importance

5.1 Implementation

I implemented a permutation importance algorithm to the decision tree. This algorithm was largely inspired by the scit-learn user guide here: https://scikit-learn.org/stable/modules/permutation_importance.html#outline-of-the-permutation-importance-algorithm

The core idea behind the algorithm is: «What happens to the model if we break of a feature?». You have a trained model and a held-out set. Score it normally, that's the reference score s. Now take one feature's column and shuffle it randomly among the rows. Every value that was in the column is still there, so the column's distribution is unchanged; what's destroyed is the *pairing* between that feature and the label. Row 7 now has some other customer's tenure.

Score the model again on this corrupted data. If the feature mattered, predictions get worse and the score drops. If the model never really used it, shuffling changes nothing.

The drop is the importance:

$$i_j = s - \frac{1}{K}\sum_{k=1}^{K} s_{k,j}$$

where $s_{k,j}$is the score on the k-th shuffle of feature j. You repeat K times because a single shuffle is one random draw, it might happen to be unusually benign or unusually damaging. Averaging over 30 gives a stable estimate, and the spread across those 30 tells you how much to trust it.

Every other feature keeps its true relationship to the label. That's what isolates feature j's contribution, you're asking, "what does the model lose when *this* feature becomes noise, given it still has everything else?"

The model is never retrained. This is what makes it cheap: $K \times d$ predictions instead of $K \times d$ model fits. But it also defines what's being measured. You're not asking, "how useful is this feature in principle?", you're asking, "how much does *this already-trained model* rely on it?" Those are different questions and conflating them, is the source of most misreadings of the output.

5.2 Results

Values for recreation	
Reference accuracy (unpermuted)	0.7285
n_repeats	30
Random seed	42

	feature	***importance_mean***	***importance_std***
0	OnlineSecurity	0.042739	0.008153
1	Contract	0.042597	0.007354
2	gender	0.000000	0.000000
3	DeviceProtection	0.000000	0.000000
4	MonthlyCharges	0.000000	0.000000
5	PaymentMethod	0.000000	0.000000
6	PaperlessBilling	0.000000	0.000000
7	StreamingMovies	0.000000	0.000000
8	StreamingTV	0.000000	0.000000
9	TechSupport	0.000000	0.000000
10	OnlineBackup	0.000000	0.000000
11	SeniorCitizen	0.000000	0.000000
12	InternetService	0.000000	0.000000
13	MultipleLines	0.000000	0.000000
14	PhoneService	0.000000	0.000000
15	tenure	0.000000	0.000000

16	Dependents	0.000000	0.000000
17	Partner	0.000000	0.000000
18	TotalCharges	0.000000	0.000000

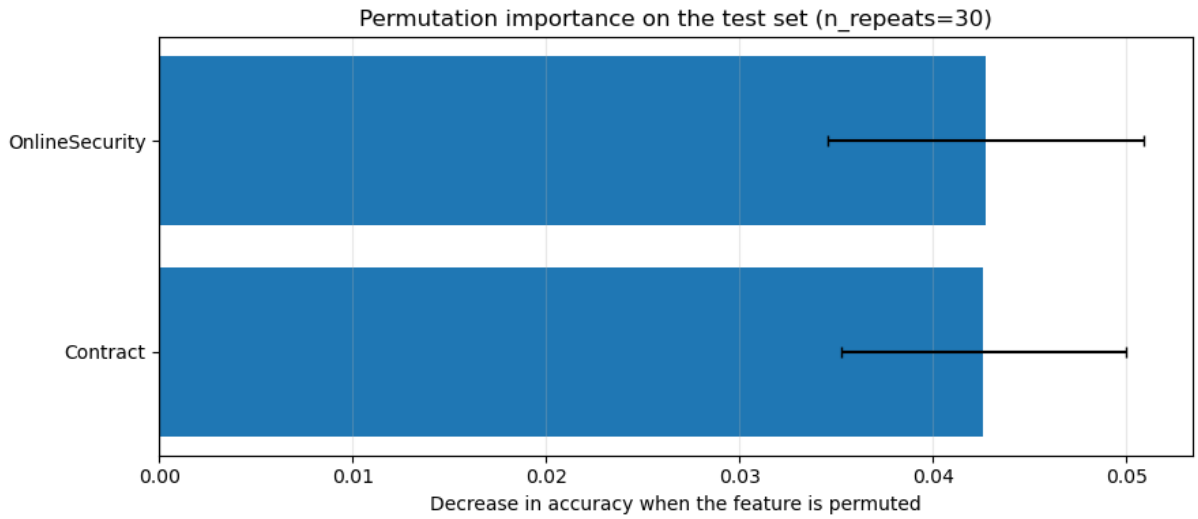


Figure 2: Permutation importance with std represented by black line

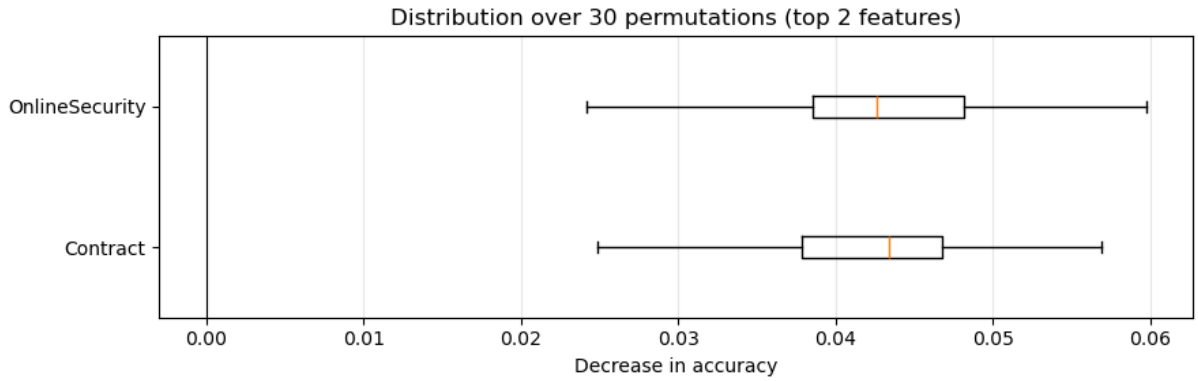


Figure 3: Decrease in accuracy of the model when corrupting OnlineSecurity and Contract

5.3 What the scores tell you

From the results above I can see that onlineSecurity and contract were the only two features to have an effect on our model. This does not mean that alle the other features that scored 0 on the permutation importance carry no meaning to the target, but it means that our particular tree does not make decisions based on them. An example of a feature scoring 0 but still being in the tree is the fact that the tree's third split is on MonthlyCharges, and both its children predict class 0. The split is inert. That's why MonthlyCharges scores 0 despite being in the tree, a third category beyond used and unused.

The features that our model does use represent whether they are a customer of the online security of the company and what type of (if any) binding contract the customer has respectively. It was a given that the contract feature was a good candidate for the model, but the online security feature was slightly more surprising. It could be that the encoding of the feature plays an as important role as the feature itself. The different values that the feature can take on are: “Yes”, “No” and “No internet service”. It could be that the “No internet service” value is what the model is actually using here, and not whether they have online security or not. This could be solved via a different method for encoding.

5.4 Weaknesses

The permutation importance is a very useful method for determining how much the model uses each feature, but it does have some flaws. The statistic can mislead you into believing that all the features with value 0 are completely unimportant and contain no relevant information. This is not the case. The rest of the values could also possess useful information, but the model does not make decisions based on them. Our model could only rank 3 features (MonthlyCharges has no importance as mentioned in 5.3, and thus has a score of 0 as well), so a deeper tree could find more information about the usefulness of different features. Correlated features also depress each other's scores. When one of a correlated pair is permuted, the model can still recover most of that information from the other, so the drop in accuracy underestimates how much the feature actually contributes.

---

6. Conclusion

In this project I have implemented a decision tree, found the optimal values for it given my data, evaluated it and looked at the feature importance. My model is not perfect. There are many improvements I could make. I could have tested far more hyperparameters and perhaps not included both GINI and Entropy, but rather something more different given their similarities as mentioned in 3.7. The encoding could have been done in a better way to avoid the problem I mentioned with the “No internet service” customers in 5.3. There is also the fact that the model performed no differently than the premade one, meaning that I put a lot of effort into something I could have done with a premade model with no benefit (outside from the academic benefits and added flexibility in the model of course).

The mode with a depth of 2 and entropy as its criterion, got a F1 score of 0.5784, a respectable score for such a simple model on this task. The two features my model based its result on were contract and onlinesecurity.